

1. What are Supervised Learning Algorithms?

Supervised Learning is one of the most widely used types of Machine Learning in which a model learns from historical data that already contains both input features and their corresponding correct outputs. The main objective of supervised learning is to discover the relationship between input variables and output variables so that it can make accurate predictions on new, unseen data. During training, the algorithm compares its predictions with actual values, calculates the error, and continuously improves itself. Supervised learning problems are generally divided into two categories: Regression and Classification. Regression is used when the output is a continuous value, while Classification is used when the output belongs to a category.

supervised learning is used in house price prediction, weather forecasting, stock market analysis, disease diagnosis, fraud detection, email spam filtering, and recommendation systems. For example, if a company has historical data containing house area, number of bedrooms, and house prices, a supervised learning algorithm can learn these patterns and predict the price of a new house.

Example :

```
from sklearn.linear_model import LinearRegression
```

```
X = [[1000], [1500], [2000]]
```

```
y = [30, 45, 60]
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
print(model.predict([[1800]]))
```

2. Difference Between Regression Algorithms and Classification Algorithms and Classification Algorithms?

Regression and Classification are the two major categories of supervised learning algorithms. The primary difference lies in the type of output they predict. Regression algorithms are used when the target variable is continuous and numerical. They predict exact values such as house prices, sales revenue, temperature, or stock prices. Classification algorithms, on the other hand, are used when the target variable belongs to predefined categories or classes. They predict labels such as Yes/No, Pass/Fail, Spam/Not Spam, or Disease/No Disease.

For example, if a real estate company wants to predict the selling price of a house, Regression algorithms such as Linear Regression are used because the output is a numerical value. However, if a bank wants to predict whether a customer will default on a loan or not, Classification algorithms such as Logistic Regression or Decision Trees are used because the output belongs to a category.

In practical machine learning projects, selecting the correct type of algorithm is important because using a regression model for a classification problem or vice versa can produce inaccurate results.

Regression Example:

Predicting house price = ₹50,00,000

Classification Example:

Predicting email = Spam or Not Spam

3. Explain Simple Linear Regression

Simple Linear Regression (SLR) is a statistical and machine learning technique used to model the relationship between one independent variable and one dependent variable. It assumes that the relationship between the variables is linear, meaning that a change in the input variable causes a proportional change in the output variable. The goal of Simple Linear Regression is to find the best-fitting straight line through the data points.

The mathematical equation of Simple Linear Regression is:

$$Y=mX+c$$

where Y is the dependent variable, X is the independent variable, m represents the slope of the line, and c represents the intercept. The slope indicates how much the output changes when the input increases by one unit.

In a practical scenario, suppose we want to predict a student's marks based on the number of hours studied. If students who study more hours generally score higher marks, the algorithm can learn this relationship and estimate marks for new students based on their study hours.

Simple Linear Regression is widely used because it is easy to understand, computationally efficient, and serves as the foundation for more advanced regression techniques.

Example :

```
from sklearn.linear_model import LinearRegression
```

```
import numpy as np
```

```
X = np.array([1,2,3,4]).reshape(-1,1)
```

```
y = [30,40,50,60]
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
print(model.predict([[5]]))
```

4. List and Explain the Assumptions of Linear Regression.

Linear Regression relies on several assumptions to provide accurate and reliable predictions. If these assumptions are violated, the model's performance and interpretation may become misleading.

The first assumption is **Linearity**, which states that there should be a linear relationship between independent and dependent variables. For example, as the area of a house increases, its price should generally increase in a linear manner.

The second assumption is **Independence of Errors**, which means that the prediction errors of one observation should not influence another observation. This is especially important in time-series data where observations are collected sequentially.

The third assumption is **Homoscedasticity**, which means that the variance of errors should remain constant across all levels of the independent variable. If errors become larger as the predicted values increase, the assumption is violated.

The fourth assumption is **Normality of Residuals**, which states that the residuals (errors) should follow a normal distribution. This helps in statistical testing and confidence interval estimation.

The fifth assumption is **No Multicollinearity**, which means that independent variables should not be highly correlated with each other. Highly correlated features provide redundant information and can make coefficient estimates unstable.

In practical machine learning projects, these assumptions are checked using scatter plots, residual plots, histograms, correlation matrices, and Variance Inflation Factor (VIF) analysis before finalizing the model.

5. What is Bias-Variance Trade-Off?

The Bias-Variance Trade-Off is one of the most important concepts in Machine Learning because it explains the balance between model simplicity and model complexity. A good machine learning model should generalize well to unseen data, and achieving this requires controlling both bias and variance.

Bias refers to the error caused by overly simplistic assumptions in the model. High bias models fail to capture the underlying patterns in the data and often lead to underfitting. For example, fitting a straight line to highly nonlinear data may result in poor predictions because the model is too simple.

Variance refers to the model's sensitivity to training data. High variance models learn not only the actual patterns but also the noise and outliers present in the dataset. Such models perform extremely well on training data but poorly on testing data, leading to overfitting.

The goal is to find the optimal balance where both bias and variance are minimized. This balance is known as the Bias-Variance Trade-Off. In practical machine learning projects, techniques such as cross-validation, regularization, feature selection, and hyperparameter tuning are used to achieve this balance.

For example, in a house price prediction project, a very simple linear model may underfit the data, while a highly complex polynomial model may overfit it. The best model lies somewhere in between.

6. Explain Overfitting and Underfitting with Examples

Overfitting and Underfitting are common problems encountered during machine learning model development. Both affect the model's ability to generalize to new data.

Overfitting occurs when a model learns not only the actual patterns in the training data but also the noise and random fluctuations. As a result, the model achieves very high accuracy on training data but performs poorly on unseen testing data. Overfitting usually happens when the model is too complex, contains too many features, or is trained for too long. In a practical scenario, it is similar to a student memorizing answers from previous exams without understanding the concepts. When different questions appear in the final exam, the student struggles to answer them correctly.

Example:

- **Training Accuracy = 99%**
- **Testing Accuracy = 70%**

This large difference indicates overfitting.

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It performs poorly on both training and testing datasets because it has not learned enough information from the data. Underfitting is similar to a student studying only one chapter before an exam and expecting to answer all questions correctly.

Example:

- **Training Accuracy = 60%**
- **Testing Accuracy = 58%**

Both values are low, indicating underfitting.

In practical machine learning projects such as house price prediction, overfitting can be reduced using regularization, feature selection, cross-validation, and collecting more data. Underfitting can be addressed by adding more relevant features, increasing model complexity, or using more advanced algorithms.

A good model should have similar and high training and testing performance. For example:

- Training $R^2 = 0.92$
- Testing $R^2 = 0.90$

This indicates that the model has learned meaningful patterns and generalizes well to unseen data.